GC EDUCATION ANALYTICS WORKBOOK

GC Apps Script Prompt Library for Schools

Google Apps Script + AI prompting for school teams

Build useful internal tools without turning student data into AI training material. This guide gives you copy/paste prompts, UI language, deployment reminders, and safety checks for school workflows built with Google Sheets, Apps Script, HTML, CSS, Chart.js, and Gmail drafts.

Best used for
dashboards, request forms, admin panels, email queues, role-based web apps, triggers, workflow tools, and clean interfaces for non-technical staff.

How this workbook is different
This is not a coding textbook. It is a prompting field guide for school leaders, counselors, office teams, and instructional leaders who know the workflow they need but do not want to become full-time developers. The pattern is simple: explain the school problem, describe safe sample data, ask AI for the next piece, test with fake rows, then improve the tool in small passes.

The working rule
Prompt AI. Copy/paste carefully. Test with fake data. Deploy only when the workflow is safe. If the tool sends emails, changes records, exposes student information, or grants access to staff, build a review step first.

Contents

1 Quick Start

THE SIMPLE LOOP Prompt AI, copy/paste, deploy, improve

1. Describe	Tell AI the workflow in plain school language. Name the audience, the Google Sheet structure, and the exact result you want.
2. Prompt	Ask for one build step at a time: plan, file structure, Apps Script code, HTML page, CSS styling, Chart.js chart, modal, email draft, trigger, or deployment checklist.
3. Copy/paste	Put each file where AI tells you: `Code.gs`, `Index.html`, `Styles.html`, `JavaScript.html`, or other Apps Script files.
4. Test	Use fake rows and fake users. Confirm filters, permissions, button actions, and email drafts before using real workflows.
5. Improve	Ask AI for polish: clearer labels, less text, better mobile layout, loading states, error messages, and role-specific pages.

Safety note: Use safe sample data

Do not paste real student data, staff data, parent data, grades, behavior notes, IEP information, health information, or private records into AI. Describe structure, fake examples, and expected behavior instead. Give AI the column headers, 2 or 3 fake example rows, and a description of the logic. Example: "My sheet has columns StudentID, GradeLevel, AdvisorEmail, RiskLevel, LastContactDate. Use fake rows only."

Copy/paste prompt: starter project

I work at a school and want to build a Google Apps Script web app connected to a Google Sheet. Please help me in small steps. Start by asking me for the sheet names, column headers, user roles, and the main actions staff need. Do not ask me to paste real student data. Use fake sample rows only. After you understand the workflow, give me a simple build plan with files for Code.gs, Index.html, Styles.html, and JavaScript.html.

2 First Scenario

FROM SHEET TO SCHOOL TOOL Build a simple workflow app, then make it feel professional

Scenario: a school team has a Google Sheet of student follow-up tasks. Staff need a cleaner web page where they can filter records, update status, write notes, and prepare family emails. The first version should be safe and boring. The later versions can add polish.

Version 1: functional draft

Ask AI for a simple Apps Script web app that reads fake rows from a sheet, displays them in a table, lets staff filter by grade or status, and updates a status column. Do not ask for advanced styling yet. The first goal is to make the data flow work.

Copy/paste prompt: first useful app

Build a Google Apps Script web app for a school follow-up tracker. Use these fake sheet details: Sheet name: FollowUps. Columns: StudentID, GradeLevel, AdvisorEmail, RiskLevel, LastContactDate, FollowUpStatus, Notes. I need a web page with filters for GradeLevel, RiskLevel, and FollowUpStatus, a table of results, and a button to update FollowUpStatus and Notes. Please give me complete Code.gs, Index.html, Styles.html, and JavaScript.html files. Use fake data examples only. Include comments showing where I replace the spreadsheet ID.

Version 2: visual upgrade

Once the workflow works, prompt for a cleaner interface: a narrow header, simple cards, table spacing, readable status badges, and one primary action per screen. Ask for school-friendly language, not developer language.

Copy/paste prompt: make it look polished
Improve the UI without changing the data logic. Make it feel like a clean school operations dashboard: dark navy header, muted teal accent, white cards, readable table rows, small status badges, clear empty states, and buttons that are obvious but not loud. Keep the language non-technical. Add comments explaining which CSS values control color, spacing, and button style.

3 Safety First

PROTECT THE DATA BEFORE THE DESIGN **How to describe data without giving AI the data**

Most useful prompts need structure, not private records. AI can usually write the correct code if it knows sheet names, column headers, example value types, and expected rules.

Instead of real rows	Provide 2 or 3 fake rows with invented names, fake emails, and made-up IDs.
Instead of notes	Describe note categories: "short counselor note," "attendance concern," "parent contact summary."
Instead of student names	Use Student A, Student B, Student C or fake IDs like S1001.
Instead of exact school data	Share the headers, value examples, and rules: "RiskLevel can be Low, Medium, High."
Instead of private links	Say "I will replace `SPREADSHEET_ID` later."

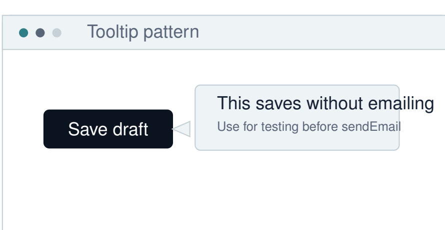
Copy/paste prompt: safe sheet description
I cannot paste real school data. Here is the safe structure only. Spreadsheet has tabs: Students, Interventions, Staff. Students headers: StudentID, GradeLevel, CounselorEmail, RiskLevel, LastUpdate. Interventions headers: StudentID, InterventionType, StartDate, Status, OwnerEmail. Staff headers: Email, Role, GradeLevelsAllowed. Use fake sample rows and write Apps Script code that I can connect to my real spreadsheet later.

Safety note: before GmailApp.sendEmail
Never start with live sending. Ask AI for draft mode, preview mode, or email queue mode first. Build a step where a human reviews the message, recipient, subject, and attachments before anything sends.

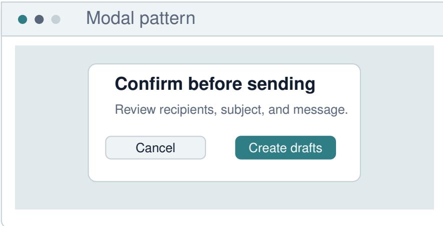
4 Visual UI Pattern Gallery

WHAT TO ASK FOR WHEN YOU WANT THE TOOL TO FEEL MODERN **Pictures, patterns, and prompts**

The fastest way to get a better interface is to name the UI pattern. You do not need technical language. You can say, "Show help text when someone hovers," "open a confirmation window," or "show a chart card." The examples below give you both the plain-language idea and a prompt.



Tooltip. A small help bubble that explains a button, field, or status without adding more text to the page.
Prompt: Add tooltips to important buttons and labels. Use short school-friendly help text. Tooltips should appear on hover and keyboard focus. Make sure they do not cover the button on small screens.



Modal. A centered confirmation window. Use it before deleting records, sending emails, changing permissions, or submitting sensitive notes.
Prompt: Add a modal confirmation before the action runs. The modal should summarize what will happen, show the record count, and offer Cancel and Confirm buttons. Keep the background dimmed and return focus to the original button when closed.

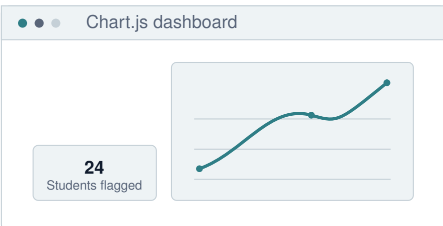
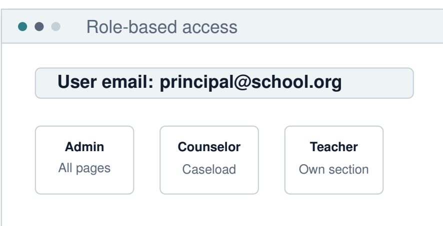
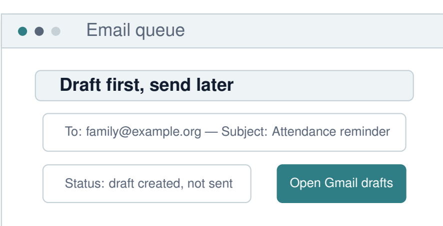


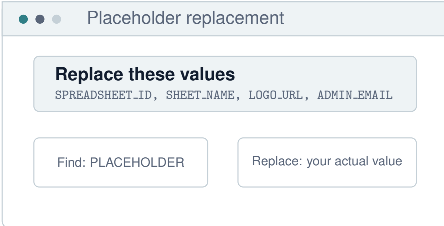
Chart.js dashboard. A visual summary card for trends, counts, categories, or status over time.
Prompt: Add Chart.js charts to this Apps Script web app. Include one line chart for weekly counts and one bar chart by category. Use fake values first. Put the chart code in JavaScript.html and explain which data shape the server should return.



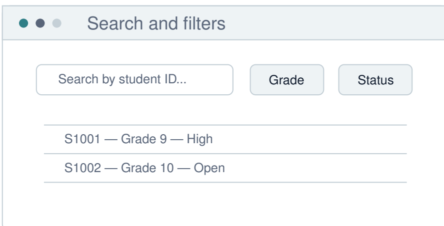
Role-based access. A page that changes based on the signed-in user's email or role.
Prompt: Add role-based access. Staff should only see records allowed for their role. Admins see all pages. Counselors see assigned grade levels. Teachers see only their own section. Also protect server-side functions so unauthorized users cannot request hidden data directly.



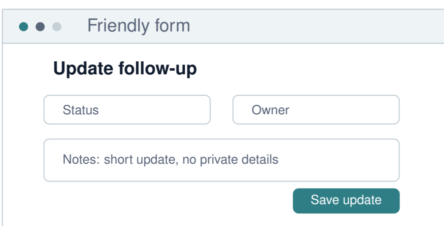
Email queue. A safer email workflow where the app creates drafts or queues messages for review before sending.
Prompt: Replace direct sending with an email review queue. The app should generate Gmail drafts first, show a preview table, and only send after an admin confirms. Include a `TEST_MODE` variable so I can keep all emails going to my own address while testing.



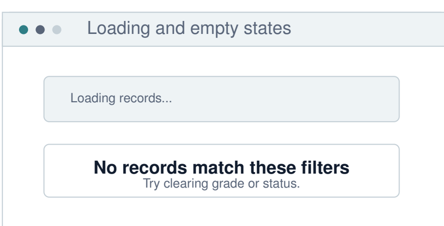
Placeholder replacement. AI often gives values like `SPREADSHEET_ID` or `LOGO_URL`. That is normal. Replace them carefully.
Prompt: List every placeholder in the code you gave me. For each one, explain where I find the real value, what file it belongs in, and what could break if I replace it incorrectly.



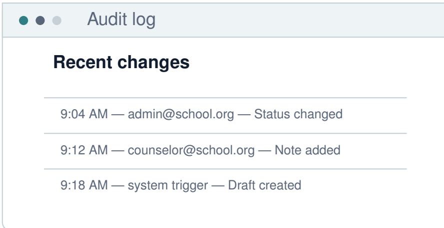
Search and filters. Staff should be able to find the right records without scrolling through a spreadsheet.
Prompt: Add a search bar, grade filter, status filter, and clear-filters button. When no records match, show a helpful empty state instead of a blank table.



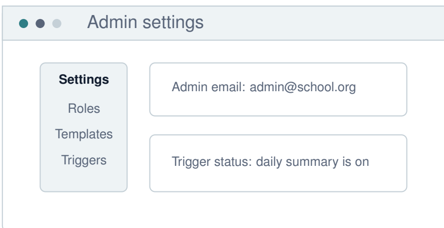
Friendly forms. A form should use school words, visible requirements, and clear confirmation after saving.
Prompt: Redesign the edit form so staff know what each field means. Add helper text, required-field validation, a success message, and a "review before submit" option for sensitive updates.



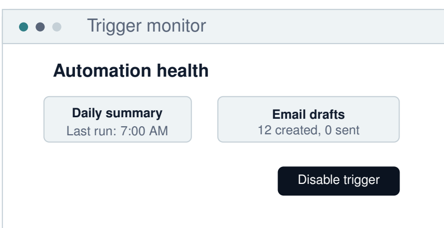
Loading and empty states. These small details make a tool feel intentional instead of broken.
Prompt: Add loading, error, empty, and success states. Use short, calm messages that tell the user what is happening and what to do next.



Audit log. For school workflows, it matters who changed what and when.
Prompt: Add an audit log tab and UI panel. Log timestamp, user email, action type, record ID, before value, after value, and status. Make the log readable for an administrator.



Admin settings. Let the owner change roles, templates, and settings without editing code.
Prompt: Add an admin settings page for roles, allowed values, email templates, app title, logo URL, and trigger status. Only Admin users should access it, and server-side permission checks must enforce that.

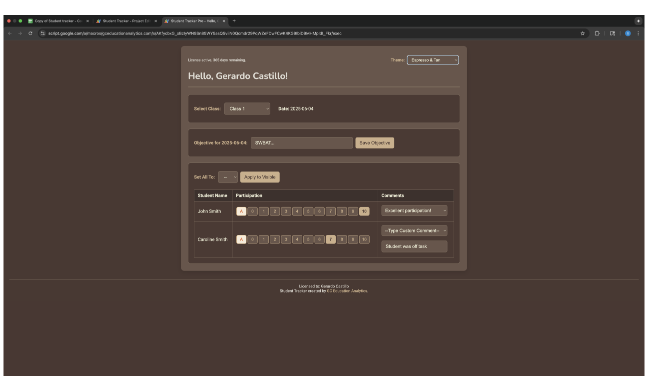


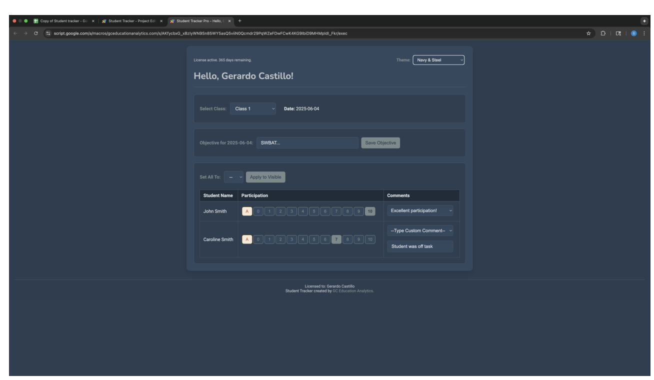
Trigger monitor. Scheduled automations need a visible off switch and a last-run status.
Prompt: Add a trigger monitor that shows active triggers, last run time, last error, and buttons to create or disable triggers. Include logs so I can tell whether automation is working.

5 Screenshot Examples

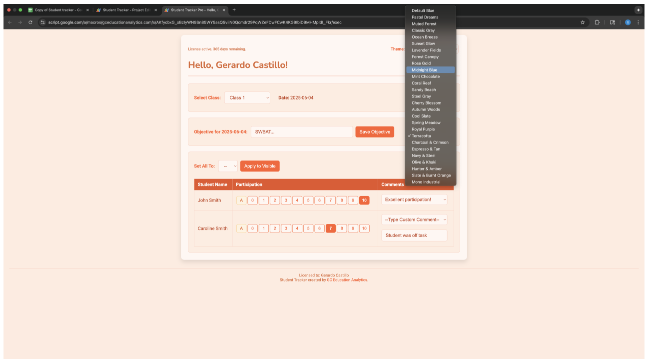
REALISTIC VISUAL REFERENCES Use screenshots as design direction, not as private data

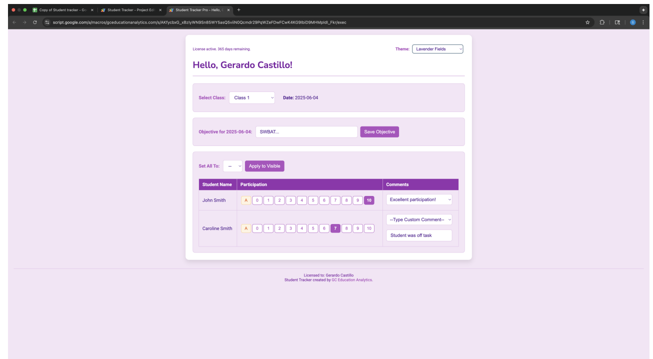
When you prompt for UI polish, you can safely describe screenshots or use generic mockups. Do not upload student data screens. If you use a screenshot, blur or remove private records first.





Prompt from a screenshot
I want my Apps Script page to feel like a polished school dashboard, similar to the attached mockup. Do not copy private data. Focus on layout: header, cards, filters, status badges, table spacing, and clear buttons. Give me updated HTML and CSS only unless JavaScript changes are required.





6 Prompt Recipes

COPY, PASTE, ADAPT Planning, data structure, UI, charts, email, triggers, security

Copy/paste prompt: project planner
Act as a Google Apps Script project planner for a school operations tool. Ask me for the purpose, user roles, sheet tabs, column headers, actions, reports, email needs, and privacy risks. Do not write code yet. After I answer, summarize the safest build plan in phases: data read, UI display, edits, email draft mode, permissions, testing, deployment.

Copy/paste prompt: column reference cheat
I have a Google Sheet where columns may move over time. Please write Apps Script helper code that maps column names to column indexes by reading the header row. Do not hard-code column numbers. Include examples for getByHeader(row, headerName) and setByHeader(row, headerName, value). Explain how this prevents errors when someone inserts a new column.

Copy/paste prompt: many sheets in one spreadsheet
My spreadsheet has multiple tabs in one Google Sheet file. Tabs are Students, Staff, Interventions, EmailQueue, Settings. Write Apps Script code that opens each tab safely by name, checks that required headers exist, and returns a clear error if a tab or header is missing. Use fake headers and show me where to edit them.

Copy/paste prompt: many spreadsheet files
My workflow uses multiple Google Sheet files, not just tabs. Please show the safest way to store each spreadsheet ID in one CONFIG object. Explain where each ID is used. Add validation so the app gives a friendly setup error if an ID is missing or points to a spreadsheet I cannot access.

Copy/paste prompt: non-technical UI/UX
Improve this interface for busy school staff. Use plain language, fewer choices per screen, visible next steps, helpful empty states, and clear success/error messages. Make the main action obvious. Use dark navy, muted teal, white cards, and soft borders. Avoid giant text, clutter, and generic AI-looking gradients.

Copy/paste prompt: modal dialogs
Add modal dialogs for three actions: update status, create email drafts, and archive a record. Each modal should summarize the action, show key details, include Cancel and Confirm buttons, and prevent accidental double-click submission. Include accessible labels and keyboard support.

Copy/paste prompt: sidebar details panel
Add a slide-in details panel when a user clicks a row. The panel should show the selected record, action history, notes, and buttons for Update Status and Create Draft Email. Keep the table visible behind it. Make it work well on mobile by turning the panel into a full-width drawer.

Copy/paste prompt: tooltip help system
Add tooltips and small help text for fields that school staff might misunderstand. Explain RiskLevel, FollowUpStatus, LastContactDate, and OwnerEmail. Tooltips should be short, readable, and accessible by keyboard. Put all tooltip text in one object so I can edit it later.

Copy/paste prompt: Chart.js cards
Add a Chart.js dashboard to this Apps Script web app. I want cards for total open items, overdue items, completed this week, and high-priority items. Add a bar chart by grade level and a line chart by week. Use fake data first, then show the server function shape that should return live chart data.

Copy/paste prompt: email draft mode
Create an email workflow that never sends immediately. It should generate a preview table first. Then it should create Gmail drafts using GmailApp.createDraft. Include a `TEST_MODE` setting where all recipient emails are replaced by my own email. Do not use GmailApp.sendEmail until I specifically ask for live sending.

Copy/paste prompt: triggers
Help me add time-based triggers safely. I need a daily check that finds overdue records and creates summary drafts for admins. Show me how to create the trigger, how to prevent duplicate triggers, how to log each run, and how to disable the trigger if something goes wrong.

Copy/paste prompt: placeholder finder
Scan the code you gave me and create a checklist of placeholders I must replace. Include spreadsheet IDs, sheet names, column names, email addresses, logo URLs, deployment URLs, admin emails, and test mode settings. For each placeholder, tell me where to find the real value.

Copy/paste prompt: logo and branding
Add my school or organization logo to the app header. Use a `LOGO_URL` placeholder first. Explain how to use an image URL from Google Drive only if it is accessible to the users. Also give me an option to store the logo as a base64 image or use a public asset URL.

Copy/paste prompt: deployment walkthrough
Walk me through deploying this Google Apps Script as a web app. Explain execute-as options, who-has-access options, editor access, versioning, and what to test before sharing the URL. Include a checklist for fake-user testing and permission testing.

Copy/paste prompt: role-based security
Add role-based access to this Apps Script web app. Use a Staff tab with Email, Role, GradeLevelsAllowed, and Active columns. The client UI should hide pages the user cannot access, but the server code must also check permissions before returning data. Explain how to prevent unauthorized users from querying hidden data by calling server functions directly.

7 Feature Library

ASK FOR FEATURES BY NAME **Small upgrades that make school tools feel finished**

Buttons	Primary action, secondary action, disabled state, loading state, confirmation state. Ask AI to make buttons say exactly what they do.
Filters	Grade, status, owner, date range, risk level, search box, saved filters. Ask for reset filters and clear empty states.
Tables	Sticky header, compact rows, status badges, row click details, pagination, export button, sort by column.
Forms	Required fields, friendly validation, helper text, review screen, submit confirmation, reset after success.
Modals	Confirm risky actions, preview email, edit one record, show warning before delete.
Tooltips	Explain jargon, show examples, reduce paragraph clutter, support keyboard focus.
Dashboards	Summary cards, trend chart, category chart, action list, last-updated timestamp.
Email queue	Create drafts, preview recipient list, test mode, send log, opt-out or skip rules.
Audit log	Timestamp, user email, action type, record ID, before value, after value.
Admin panel	Manage settings, roles, allowed values, template text, trigger status, logs.
Settings page	Spreadsheet IDs, app name, logo URL, email template, school year, default filters.
Permissions	Active users, role checks, allowed tabs, server-side restrictions, friendly denied page.

Copy/paste prompt: feature menu

Give me a menu of optional upgrades for this Apps Script school tool. Group them by usability, reporting, automation, email, permissions, and admin settings. For each upgrade, explain the benefit in plain language and tell me which files would change.

8 Email Automation

DRAFT FIRST, SEND LATER How to prompt for safer Gmail workflows

Email automation is useful but risky. The safest pattern is: create preview, create drafts, review drafts, then optionally send. If a message affects families, staff, or students, keep a human review point.

Recommended email flow

1. Generate message text from a template. 2. Show a preview table. 3. Create Gmail drafts only. 4. Log draft creation. 5. Have a staff member review drafts in Gmail. 6. Move to live sending only after repeated testing with fake rows and test recipients.

Copy/paste prompt: email templates from rows

Create an email template system for Apps Script. The sheet has fake columns: StudentFirstName, AdvisorEmail, ParentEmail, MissingItemsCount, DueDate. Use placeholders like StudentFirstName and DueDate. Generate preview messages first. Then create Gmail drafts. Include a log tab with timestamp, recipient, subject, status, and row ID.

Copy/paste prompt: live send only after review

I have tested draft mode and now want to understand live sending. Before writing GmailApp.sendEmail code, explain the risks, quota limits, test steps, logging needs, opt-out rules, and rollback plan. Then show me how to keep `TEST_MODE` available so live sending can be turned off quickly.

9 Role-Based Access

DO NOT RELY ON HIDDEN BUTTONS Protect data on the server side

Hiding a page in HTML is not security. If a user can still call a server function and receive data, the app is not protected. Ask AI to check permissions in every server function that returns or changes records.

Safety note: security phrasing to use

Use this phrase often: "Do not only hide UI elements. Add server-side permission checks before returning data or updating records."

Copy/paste prompt: permission test plan

Create a permission test plan for this Apps Script web app. Include fake users for Admin, Counselor, Teacher, and Unauthorized. For each user, list what pages they should see, what server functions they can call, what data they should receive, and what error they should get if they try an unauthorized action.

Copy/paste prompt: access denied page

Add an access denied screen that is calm and helpful. It should say the user does not have access, show the signed-in email if available, and provide a contact email for support. Do not reveal what restricted records or pages exist.

10 Deployment Checklist

BEFORE YOU SHARE THE LINK Apps Script web app launch checks

- [ ] Replace every placeholder: spreadsheet ID, sheet names, logo URL, admin email, test recipient, deployment URL, role names.
- [ ] Confirm required headers exist in every tab and the app gives friendly setup errors if not.
- [ ] Test with fake rows before real records.
- [ ] Confirm `TEST_MODE` is on for email workflows.
- [ ] Create Gmail drafts before using any live send behavior.
- [ ] Check Apps Script deployment: execute as, who has access, version, and description.
- [ ] Limit editor access to people who truly need to change code.
- [ ] Test unauthorized access with a fake user or account that should not see records.
- [ ] Confirm server-side permission checks exist, not only hidden buttons.
- [ ] Add an audit log for sensitive changes.
- [ ] Keep a backup copy of the spreadsheet before testing write actions.
- [ ] Document how to disable triggers, stop email sending, and roll back a deployment.

Copy/paste prompt: deployment checklist from my code

Review this Apps Script project for deployment readiness. Create a checklist of risks and fixes. Check placeholders, permissions, web app deployment settings, trigger behavior, email draft mode, logging, sheet headers, error messages, and editor access. Use plain language for a school admin.

11 Troubleshooting

WHEN AI GIVES YOU ALMOST-RIGHT CODE How to get unstuck without becoming the developer

Placeholder errors	Ask AI to list placeholders and explain exactly where to replace each one.
Blank page	Ask for browser console checks, Apps Script execution logs, and a minimal test page.
No data showing	Ask AI to verify spreadsheet ID, sheet name, headers, permissions, and returned JSON shape.
Button does nothing	Ask AI to trace the click handler, client function, server function, and success/error callback.
Email risk	Ask AI to convert live send into draft mode and add `TEST_MODE`.

Access issue	Ask AI to show current user email, role lookup, and server-side deny logic.
Ugly interface	Ask for one design pass only: spacing, font sizes, cards, colors, labels, mobile layout.
Too much code	Ask AI to split code by file and tell you exactly where each file goes.

Copy/paste prompt: explain where code goes

I am non-technical. Explain exactly where each piece of code goes in Google Apps Script. Tell me which file to create, what to paste, what to replace, and what to test first. If there are placeholders, list them in a checklist.

Copy/paste prompt: debug with logs

Help me debug this Apps Script app step by step. Add clear Logger.log statements on the server and console.log statements on the client. Tell me where to view each log. Do not rewrite the whole project unless necessary.

12 Appendix A: Prompt Bank

FAST STARTERS Short prompts grouped by goal

Planning

Ask me one question at a time so we can design this school workflow safely. Start with the problem, then users, sheet tabs, headers, actions, permissions, emails, and deployment.

Data

Use header names instead of fixed column numbers. Create helper functions for reading and writing by header name. Include friendly errors for missing headers.

UI

Make the page feel premium and calm: dark navy header, muted teal accent, white cards, compact typography, useful empty states, and clear actions. Avoid giant text and clutter.

Modal

Add a confirmation modal before this action. It should explain what will happen, show key details, and include Cancel and Confirm buttons.

Tooltip

Add tooltips for confusing fields. Keep each tooltip under 14 words. Make them work by hover and keyboard focus.

Chart.js

Use Chart.js to create one line chart and one bar chart. Start with fake data. Explain the data format needed from Apps Script.

Email

Create Gmail drafts only. Add `TEST_MODE`. Log every draft. Do not use GmailApp.sendEmail yet.

Trigger

Create a daily trigger safely. Prevent duplicate triggers. Add a disable function. Log each run.

Security

Add server-side role checks to every function that returns or updates records. Do not rely on hidden buttons.

13 Appendix B: Column and Sheet Cheat Sheet

STRUCTURE AI CAN USE How to explain your spreadsheet

Single tab	”My spreadsheet has one tab named FollowUps. Headers are...”
Many tabs	”Tabs are Students, Staff, Interventions, EmailQueue, Settings. Join records by StudentID.”
Many files	”There are separate spreadsheet files. Store IDs in CONFIG and explain each ID.”
Header row not row 1	”Headers start on row 3. Data starts on row 4. Ignore rows above headers.”
Merged/sectioned sheet	”The sheet has multiple mini-tables on one tab. Ask AI to recommend separating them into clean tabs before coding.”
Changing columns	”Use header lookup, not fixed column indexes.”
Allowed values	”RiskLevel can be Low, Medium, High. Status can be Open, Drafted, Completed, Archived.”
Private values	”Use fake IDs and fake emails. Do not paste real records.”

Copy/paste prompt: messy sheet redesign
My current sheet has several small tables on one tab and it is hard to automate. Please recommend a cleaner tab structure for Apps Script. Use normalized tabs if helpful. Explain the new tabs, headers, unique IDs, and how the web app would read them. Keep it understandable for a school office team.

14 Appendix C: Ask AI to Improve This

META-PROMPTS Use AI as a reviewer

Copy/paste prompt: review for safety
Review this Apps Script project for school data safety. Look for real data exposure, weak permissions, missing server-side checks, risky email sending, overbroad editor access, missing logs, and confusing deployment settings. Give me a prioritized fix list in plain language.

Copy/paste prompt: review for usability
Review this interface as if you are a busy school staff member with 3 minutes between meetings. Identify confusing labels, too many choices, missing help text, weak buttons, unclear errors, and mobile problems. Give me specific copy changes and UI changes.

Copy/paste prompt: review for maintainability
Review this Apps Script code for maintainability. Suggest ways to centralize config, avoid repeated code, use header-based column lookup, separate UI from server logic, and document deployment steps. Explain each recommendation for a non-technical owner.

Final reminder
The goal is not to make AI ”do school data.” The goal is to make AI help you build safer, clearer tools around workflows you already understand. Keep private records out of prompts, test in drafts, protect access on the server, and improve the interface one practical step at a time.